


PRISE EN COMPTE DE MULTI-LANGAGES (LOCALIZATION)

Introduction

Les fonctions de localisation de Laravel offrent un moyen pratique de récupérer des chaînes de caractères dans différentes langues, ce qui vous permet de prendre facilement en charge plusieurs langues dans votre application.

Laravel propose deux façons de gérer les chaînes de traduction. Premièrement, les chaînes de langue peuvent être stockées dans des fichiers situés dans le répertoire **lang**. Dans ce répertoire, il peut y avoir des sous-répertoires pour chaque langue prise en charge par l'application.

Introduction

Par défaut, les versions récentes de Laravel n'incluent pas le dossier **lang**. Vous devez le générer en utilisant la commande Artisan suivante :

```
php artisan lang:publish
```

Introduction

Voilà l'approche utilisée par Laravel pour gérer les chaînes de traduction des fonctionnalités intégrées de Laravel, telles que les messages d'erreur de validation :

```
/lang  
  /en  
    messages.php  
  /es  
    messages.php
```

Introduction

Les chaînes de traduction peuvent aussi être définies dans des fichiers **JSON** placés dans le répertoire **lang**. Avec cette approche, chaque langue prise en charge par votre application aura un fichier **JSON** correspondant dans ce répertoire. Cette approche est recommandée pour les applications qui ont un grand nombre de chaînes traduisibles.

```
/lang  
  en.json  
  es.json
```

Configurer le Locale

La langue par défaut de votre application est stockée dans l'option de configuration **locale** du fichier de configuration **config/app.php**. Vous êtes libre de modifier cette valeur pour l'adapter aux besoins de votre application.

Vous pouvez modifier la langue par défaut pour une seule requête HTTP au moment de l'exécution en utilisant la méthode **setLocale** fournie par la façade App.

```
App::setLocale($locale); // $locale = 'fr'
```

Configurer le Locale

Vous pouvez configurer une "langue de secours", qui sera utilisée lorsque la langue active ne contient pas une chaîne de traduction donnée. Comme la langue par défaut, la langue de secours est également configurée dans le fichier de configuration **config/app.php** :

```
'fallback_locale' => 'en',
```

Détermination de la localité actuelle

Vous pouvez utiliser les méthodes **currentLocale** et **isLocale** de la façade **App** pour déterminer la locale actuelle ou vérifier si la locale est une valeur donnée :

```
use Illuminate\Support\Facades\App;  
  
$locale = App::currentLocale();  
  
if (App::isLocale('en')) {  
    //  
}
```


DÉFINITION DES CHAÎNES DE TRADUCTION

+

•

○

+

○

•

Utilisation des clés courtes

En général, les chaînes de traduction sont stockées dans des fichiers situés dans le répertoire **lang**. Dans ce répertoire, il doit y avoir un sous-répertoire pour chaque langue prise en charge par votre application.

Tous les fichiers de langue renvoient un tableau de chaînes de caractères. Par exemple :

```
// lang/en/messages.php

return [
    'welcome' => 'Welcome to our application!',
];
```

Utilisation des clés de chaînes de traduction

Pour les applications comportant un grand nombre de chaînes traduisibles, le fait de définir chaque chaîne par une "**clé courte**" peut être source de confusion lorsque vous faites référence aux clés dans vos vues. Il est en outre fastidieux d'inventer continuellement des clés pour chaque chaîne de traduction prise en charge par votre application.

C'est pourquoi Laravel permet également de définir des chaînes de traduction en utilisant la traduction "par défaut" de la chaîne comme clé.

Utilisation des clés de chaînes de traduction

Les fichiers de traduction qui utilisent des chaînes de traduction comme clés sont stockés sous forme de fichiers **JSON** dans le répertoire **lang**. Par exemple, si votre application possède une traduction en **espagnol**, vous devez créer un fichier **lang/es.json** :

```
{  
    "I love programming.": "Me encanta programar."  
}
```

RÉCUPÉRATION DES CHAÎNES DE TRADUCTION

Récupération des chaînes de traduction

Vous pouvez récupérer les chaînes de traduction de vos fichiers de langue à l'aide de la fonction `__`. Si vous utilisez des "clés courtes" pour définir vos chaînes de traduction, vous devez transmettre le fichier qui contient la clé et la clé elle-même à la fonction `__` en utilisant la syntaxe **"point"**.

Par exemple, récupérons la chaîne de traduction **welcome** dans le fichier de langue **lang/en/messages.php** :

```
echo __('messages.welcome');
```

Récupération des chaînes de traduction

Si la chaîne de traduction spécifiée **n'existe pas**, la fonction `__` renvoie la **clé** de la chaîne de traduction. Ainsi, dans l'exemple ci-dessus, la fonction `__` renvoie **messages.welcome** si la chaîne de traduction n'existe pas.

Si vous utilisez vos chaînes de traduction par défaut comme clés de traduction, vous devez transmettre la traduction par défaut de votre chaîne à la fonction `__` ;

```
echo __('I love programming.');
```

Récupération des chaînes de traduction

Là encore, si la chaîne de traduction n'existe pas, la fonction `__` renvoie la clé de la chaîne de traduction qui lui a été donnée.

Si vous utilisez le moteur de création de modèles **Blade**, vous pouvez utiliser la syntaxe `{{ }} echo` pour afficher la chaîne de traduction :

```
{{ __('messages.welcome') }}
```


Paramètres des chaînes de traduction

Si vous le souhaitez, vous pouvez définir des caractères de remplacement dans vos chaînes de traduction. Tous les caractères de remplacement sont préfixés par un `:`.

Par exemple, vous pouvez définir un message de bienvenue avec un nom de caractère générique :

```
'welcome' => 'Welcome, :name',
```

Paramètres des chaînes de traduction

Pour remplacer les caractères de remplacement lors de la récupération d'une chaîne de traduction, vous pouvez passer un tableau de remplacements comme deuxième argument de la fonction `__` :

```
echo __('messages.welcome', ['name' => 'dayle']);
```

Paramètres des chaînes de traduction

Si votre caractère de remplacement contient toutes les majuscules, ou si seule la première lettre est en majuscule, la valeur traduite sera en majuscule en conséquence :

```
'welcome' => 'Welcome, :NAME', // Welcome, DAYLE  
'goodbye' => 'Goodbye, :Name', // Goodbye, Dayle
```

Pluralisation

Vous pouvez même créer des règles de pluralisation plus complexes qui spécifient des chaînes de traduction pour plusieurs plages de valeurs :

```
'apples' => '{0} There are none | [1,19] There are some | [20,*] There are many',
```

Pluralisation

Après avoir défini une chaîne de traduction qui comporte des options de pluralisation, vous pouvez utiliser la fonction **trans_choice** pour récupérer la ligne pour un "nombre" donné. Dans cet exemple, comme le nombre est supérieur à un, la forme plurielle de la chaîne de traduction est retournée :

```
echo trans_choice('messages.apples', 10);
```

Pluralisation

Vous pouvez également définir des attributs de type **placeholder** dans les chaînes de pluralisation. Ces caractères de remplacement peuvent être remplacés en passant un tableau comme troisième argument à la fonction **trans_choice** :

```
'minutes_ago' => '{1} :value minute ago | [2,*] :value minutes ago',  
echo trans_choice('time.minutes_ago', 5, ['value' => 5]);
```